

Ansible

Gestión de la configuración y automatización

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI · Puesta en Producción de Aplicaciones

PI · ANSIBLE

01

Introducción a Ansible

Qué es, instalación y primeros pasos

¿Qué es Ansible?

*"**Ansible** es una herramienta de **gestión de configuración y automatización** que permite definir, en ficheros de texto **YAML**, el estado deseado de servidores y aplicaciones, y aplicarlo **sin instalar agentes** en los nodos (usa **SSH**)."*

CARACTERÍSTICAS

- Desarrollado principalmente por **Red Hat**
- Escrito en **Python** · disponible en PyPI
- Arquitectura **push** sin agentes — solo SSH
- Configuración declarativa en **YAML**
- Primera versión: **2012**

¿PARA QUÉ SIRVE?

- Configurar servidores de forma automática
- Instalar y gestionar paquetes y servicios
- Desplegar aplicaciones
- Orquestar tareas en **múltiples máquinas** a la vez
- Gestionar actualizaciones del sistema

Ansible en el contexto de IaC

La **Infraestructura como Código (IaC)** define y gestiona la infraestructura mediante ficheros de configuración en lugar de hacerlo manualmente. Existen dos tipos de herramientas:

SOFTWARE DE ORQUESTACIÓN

Programa la **creación de escenarios**: máquinas virtuales, redes, almacenamiento...

Ejemplo: OpenTofu

SOFTWARE DE GESTIÓN DE LA CONFIGURACIÓN (CMS)

Programa la **configuración que van a tener los servidores** una vez creados.

Ejemplo: Ansible

Instalación de Ansible

CON APT (DEBIAN/UBUNTU)

```
apt install ansible
```

Instala la versión empaquetada por la distribución.

CON PIP EN ENTORNO VIRTUAL (RECOMENDADO)

```
python3 -m venv ansible-env  
source ansible-env/bin/activate  
pip install ansible
```

Permite tener la **versión más reciente** e independiente del sistema.



Ansible solo necesita instalarse en el **nodo de control** (tu máquina). Los nodos gestionados únicamente requieren Python y acceso SSH.

El inventario

El **inventario** es el fichero donde definimos los equipos que Ansible va a gestionar. Los equipos se organizan en **grupos**.

```
all:
  children:
    servidores:
      hosts:
        nodo1:
          ansible_ssh_host: 192.168.1.10
          ansible_ssh_user: usuario
          ansible_ssh_private_key_file: ~/.ssh/id_rsa
```

- `all` — grupo raíz que engloba todos los equipos
- `servidores` — grupo personalizado con nuestros hosts
- Cada host puede tener variables propias: IP, usuario SSH, clave privada...

Fichero de configuración

Junto al inventario se crea un fichero `ansible.cfg` que configura el comportamiento de Ansible para el proyecto:

```
[defaults]
inventory = hosts
host_key_checking = False
```

- `inventory` — ruta al fichero de inventario
- `host_key_checking = False` — evita la verificación de la clave del host en cada conexión SSH (útil en entornos de pruebas)



Ansible busca este fichero en el directorio de trabajo. Cada proyecto puede tener su propia configuración.

Módulos y comandos ad-hoc

Un **módulo** permite ejecutar una acción concreta en uno o varios servidores remotos.

Los **comandos ad-hoc** ejecutan un módulo directamente desde la línea de comandos, sin necesidad de un playbook:

```
ansible <hosts> -m <módulo> -a "<parámetros>"
```

PARTE	DESCRIPCIÓN
<hosts>	<code>all</code> , nombre de grupo o nombre de host del inventario
-m <módulo>	módulo a usar
-a "<parámetros>"	argumentos del módulo
--become	ejecutar como <code>root</code> en el nodo remoto

Módulos esenciales (I)

PING

Comprueba la conectividad con los nodos. No es un ping ICMP, sino una verificación de conexión SSH y Python.

```
ansible all -m ping
ansible servidores -m ping
```

COMMAND / SHELL

Ejecuta comandos en el nodo remoto. `shell` permite pipes, redirecciones y variables de entorno.

COPY

Copia ficheros desde el nodo de control al nodo remoto.

- `src` — fichero origen (local)
- `dest` — ruta de destino (remoto)
- `mode` — permisos (opcional)

```
ansible all -m copy \
  -a "src=./index.html dest=/tmp/index.html mode=0644"
```

Módulos esenciales (II)

APT

Instala, actualiza o elimina paquetes en sistemas Debian/Ubuntu.

- `name` — paquete
- `state` — `present`, `absent`, `latest`

```
ansible nodo1 -m apt \  
  -a "name=apache2 state=present" --become
```

SERVICE

Gestiona servicios del sistema.

- `name` — servicio

FILE

Gestiona ficheros, directorios y permisos.

- `path` — ruta en el nodo remoto
- `state` — `file`, `directory`, `absent`, `link`
- `mode` — permisos

```
ansible all -m file \  
  -a "path=/tmp/demo state=directory mode=0755"
```

USER

Crea, modifica o elimina usuarios.

- `name` — nombre del usuario

Declarativo e idempotencia

ENFOQUE DECLARATIVO

Ansible **no** usa un esquema imperativo (*"instala apache"*).

Declaramos el **estado deseado** del servidor:

*"Quiero que el servidor tenga
apache2 instalado"*

Ansible hará todas las operaciones necesarias para que ese estado se cumpla.

IDEMPOTENCIA

Si el estado declarado **ya se ha alcanzado**, Ansible no ejecuta ninguna operación adicional.

- Primera ejecución: instala apache → salida en **amarillo** (`changed`)
- Segunda ejecución: ya está instalado → salida en **verde** (`ok`)

Podemos ejecutar el mismo playbook **múltiples veces** con total seguridad.

PI · ANSIBLE

02

Playbooks

Automatización declarativa con Ansible

Play y Playbook

PLAY

Una **jugada** es la unidad mínima de trabajo: declara **en qué hosts** se ejecuta y **qué tareas** se realizan.

Cada tarea usa un módulo con sus parámetros para alcanzar un estado deseado.

PLAYBOOK

Un **libro de jugadas** es un fichero YAML que agrupa uno o varios plays para conseguir una **configuración completa** de la infraestructura.

Se ejecuta con:

```
ansible-playbook site.yml
```

Estructura de un playbook

```
---
- name: Configurar servidor web           # Descripción del play
  hosts: servidores                       # A qué hosts se aplica
  become: true                           # Ejecutar tareas como root (sudo)

  tasks:

    - name: Actualizar caché de apt
      ansible.builtin.apt:
        update_cache: true

    - name: Instalar Apache
      ansible.builtin.apt:
        name: apache2
        state: present

    - name: Copiar página de inicio
      ansible.builtin.copy:
        src: files/index.html
        dest: /var/www/html/index.html
        mode: "0666"
```

Variables en Ansible

Ansible puede trabajar con variables obtenidas de distintas fuentes:

NIVEL DE NODO

Variables definidas directamente en el inventario para un host concreto.

```
hosts:
  nodo1:
    ansible_ssh_host: 192.168.1.10
    ansible_ssh_user: jose
    ansible_ssh_private_key_file: ~/.ssh/id_rsa
    http_port: 80
```

NIVEL DE GRUPO

Variables en el directorio `group_vars/` que se aplican a todos los hosts de un grupo.

```
group_vars/
├── all          # Todos los hosts
└── servidores  # Solo el grupo servidores
```

Variables de grupo: group_vars

El directorio `group_vars/` contiene ficheros YAML con variables accesibles en los plays:

```
mi-proyecto/  
├─ hosts  
├─ ansible.cfg  
├─ site.yml  
└─ group_vars/  
    ├─ all          # Variables globales  
    └─ servidores  # Variables del grupo servidores
```

`group_vars/all` :

```
---  
usuario_app: deploy  
puerto_web: 80  
paquetes:  
  - apache2  
  - git
```

USO EN EL PLAYBOOK

Las variables se referencian con dobles llaves

`{{ }}` :

```
- name: Instalar paquetes  
  ansible.builtin apt:  
    name: "{{ item }}"  
    state: present  
    loop: "{{ paquetes }}"  
  
- name: Crear usuario de la app  
  ansible.builtin.user:  
    name: "{{ usuario_app }}"  
    state: present
```

Gathering Facts

Al inicio de cada play, Ansible ejecuta automáticamente la tarea **Gather Facts** que recopila información del nodo remoto.

VARIABLES DISPONIBLES

VARIABLE	VALOR DE EJEMPLO
ansible_hostname	servidor1
ansible_distribution	Debian
ansible_distribution_version	12
ansible_default_ipv4.address	192.168.1.10
ansible_memtotal_mb	2048

USO EN PLANTILLAS Y TAREAS

```
- name: Mensaje de bienvenida
  ansible.builtin.copy:
    content: "Servidor: {{ ansible_hostname }}"
  Sistema: {{ ansible_distribution }}
           {{ ansible_distribution_version }}"
  dest: /etc/motd
```

Ver todas las facts de un nodo:

```
ansible nodo1 -m setup
ansible nodo1 -m setup -a "filter=ansible_distribution*"
```

El módulo template

El módulo **template** copia una plantilla **Jinja2** al nodo remoto sustituyendo las variables por sus valores.

Las plantillas se guardan en el directorio `templates/` con extensión `.j2`:

`templates/index.j2`:

```
<!DOCTYPE html>
<html>
<body>
  <h1>{{ ansible_hostname }}</h1>
  <p>Sistema: {{ ansible_distribution }}
    {{ ansible_distribution_version }}</p>
  <p>Gestionado por: {{ usuario_app }}</p>
</body>
</html>
```

TAREA EN EL PLAYBOOK

```
- name: Desplegar página de inicio
  ansible.builtin.template:
    src: templates/index.j2
    dest: /var/www/html/index.html
    mode: "0644"
```

Jinja2 permite también **condicionales** y **bucles** dentro de las plantillas:

```
{% if ansible_distribution == "Debian" %}
Sistema compatible.
{% endif %}
```

Estructura de un proyecto Ansible

```
mi-proyecto/  
├─ ansible.cfg      # Configuración de Ansible  
├─ hosts            # Inventario  
├─ site.yml         # Playbook principal  
├─ files/           # Ficheros estáticos para copiar con copy  
│   └─ foo.txt  
├─ templates/       # Plantillas Jinja2 para el módulo template  
│   └─ index.j2  
└─ group_vars/      # Variables por grupo  
    ├─ all           # Variables globales  
    └─ servidores    # Variables del grupo servidores
```

 Ansible localiza automáticamente los directorios **files/**, **templates/** y **group_vars/** si están junto al playbook o en el directorio de trabajo.

Ejecutar un playbook e interpretar la salida

```
ansible-playbook site.yml
```

COLORES DE LA SALIDA

COLOR	ESTADO	SIGNIFICADO
 Verde	<code>ok</code>	Ya estaba en el estado deseado
 Amarillo	<code>changed</code>	Se ha realizado un cambio
 Rojo	<code>failed</code>	La tarea ha fallado
 Azul	<code>skipping</code>	Tarea omitida (condición)

RESUMEN FINAL (PLAY RECAP)

```
PLAY RECAP *****
nodo1 : ok=4  changed=2  unreachable=0
        failed=0  skipped=0
```

- `ok` + `changed` — tareas ejecutadas
- Si `changed=0` en la segunda ejecución → **idempotencia** funcionando
- `unreachable` o `failed` → hay que revisar errores

PI · ANSIBLE

¡Gracias!

Ansible · Introducción y Playbooks

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI